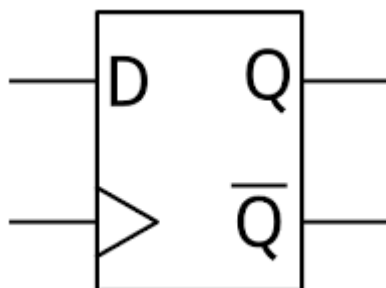


SISTEMES DIGITALS (Curs 2021-2022)

PRÀCTICA 2: Processos síncrons

Podríem dir que l'objectiu d'aquesta pràctica consisteix en aprendre a “controlar el temps” mitjançant l'ús de processos síncrons.

Un procés síncron és aquell que realitza la transferència de dades d'un registre a un altre en un instant concret, determinat per un cert esdeveniment (“event”) en un senyal de rellotge. L'exemple bàsic d'un procés síncron, seria la funció que realitza un simple flip-flop tipus D, on el senyal D és transferit a la sortida Q quan es produeix un flanc en el senyal de rellotge.



En la pràctica anterior, es va descriure una arquitectura 100% combinatòria, que bàsicament consistia en un codificador que, a partir d'una entrada de 4 bits, codificava el valor entrat en hexadecimal cap a una combinació de uns i zeros per a encendre o no els leds d'un dels 4 displays de 7 segments dels que disposa la nostra tarja Basys 3.

La primera part d'aquesta pràctica consistirà en mostrar un valor diferent en cadascun dels 4 displays. Per a això, emprarem els 16 interruptors de que disposem (4 interruptors per a cada display)

Tenint en compte que els 4 displays de la placa tenen els 8 bits dels càtodes comuns entre ells, no podem mostrar més d'un dígit alhora, per tant, si volem mostrar valors diferents en els 4 displays, hem de multiplexar en el temps el mostreig de cada un. Per a això, necessitem un procés que vagi enviant cada un dels 4 dígits al seu display corresponent de forma cíclica i a una freqüència amb la que l'ull humà no sigui capaç de notar cap canvi. D'aquesta manera, visualment podrem percebre la imatge de 4 caràcters diferents al mateix temps.

L'entitat que haurem de crear haurà de disposar dels següents ports:

```
entity prac2_1 is
  Port (  sw : in STD_LOGIC_VECTOR (15 downto 0);
         clk : in std_logic;
         reset : in std_logic;
         cat : out STD_LOGIC_VECTOR (7 downto 0);
         an : out STD_LOGIC_VECTOR (3 downto 0));
end prac2_1;
```

On, a part dels interruptors d'entrada i el control dels ànodes i càtodes dels displays de sortida, hi incorporarem un senyal de rellotge i un reset.

El primer que necessitem a la nostra arquitectura és un divisor de rellotge. L'oscil·lador de la Basys 3 genera un senyal de clk a 100Mhz que entra a la FPGA pel pin W5, i nosaltres necessitem un senyal que ens marqui el ritme de refresc dels displays a una freqüència suficientment alta per no poder percebre el canvi, però, evidentment, 100 Mhz és una mica exagerat.

Una freqüència adequada per a aquest propòsit podria ser de 1Khz. Per aconseguir aquesta freqüència haurem de dividir el senyal de rellotge per 100000, però mantenint l'ample de pols del senyal de clk original. És a dir, enlloc de dividir la freqüència mantenint el duty cycle del 50%, deixarem passar un pols de rellotge de cada 100000. El motiu d'això, és que aquest clk dividit l'utilitzarem de ENABLE en un procés síncron a la llista sensible del qual hi haurà el clk original. Tots els processos síncrons del sistema utilitzaran el mateix clk (l'original) per tal d'evitar problemes de sincronia entre ells, i el "trigger" que ens provocarà la transferència serà un senyal de l'amplada d'un sol pols de rellotge original (funció de enable en un flip-flop). En els exemples de codi següents ho podrem veure millor.

```
-- Divisor del clk (de 100Mhz a 1Khz)

divisor_clk: process (clk, reset) begin
  if clk'event and clk = '1' then
    div_clk <= '0';
    if reset = '1' then
      div_cnt <= 0;
    elsif div_cnt = 100000 then
      div_cnt <= 0;
      div_clk <= '1';
    else div_cnt <= div_cnt + 1;
    end if;
  end if;
end process;
```

Aquest procés ens generarà un pols al senyal div_clk, que tindrà l'amplada d'un període del rellotge original, i que es produirà cada 100000 polsos d'aquest últim. El senyal que utilitzarem per a comptar fins a 100000 l'haurem de declarar com a un enter amb el rang adequat.

El següent procés correspon a un altre comptador, que cada cop que rep un pols del rellotge dividit incrementa un senyal enter de 0 fins a 3. Aquest senyal ens indicarà en cada moment quin és el display que hem de refrescar.

```
digit_actual: process(clk, reset) begin
  if clk'event and clk = '1' then
    if reset = '1' then
      digit_act <= 0;
    elsif div_clk = '1' then
      if digit_act = 3 then
        digit_act <= 0;
      else digit_act <= digit_act + 1;
      end if;
    end if;
  end if;
end if;
end process;
```

A aquests dos processos, només caldria afegir-hi una sentència que seleccionés quin grup de 4 interruptors és el que hem de llegir (el que ens marqui el digit_act), un codificador d'hexadecimal a 7 segments com l'utilitzat en la pràctica anterior, i una sentència per a decidir quin dels 4 ànodes ha d'estar actiu en cada moment.

Part 2:

La segona part de la pràctica consistirà en mostrar de forma decimal en els 4 displays de 7 segments, un valor binari que entrarem amb 13 dels 16 interruptors de la placa (de 0 a 8191).

Per a això necessitarem la descripció d'una arquitectura que ens faci la conversió d'un valor binari de 13 bits a 4 BCD's (binary coded decimal) corresponents a cadascun del dígit del valor decimal a mostrar.

Busqueu per internet alguna descripció en vhdl que tingui aquesta funció (provablement la trobareu en forma de màquina d'estats), adapteu-la a la dimensió adequada, i completeu-la reutilitzant part del codi de l'apartat anterior per a mostrar els BCD's pels displays de 7 segments. Com a pista, un dels algorismes més utilitzats per a fer aquest tipus de conversió és l'anomenat Double Dabble.